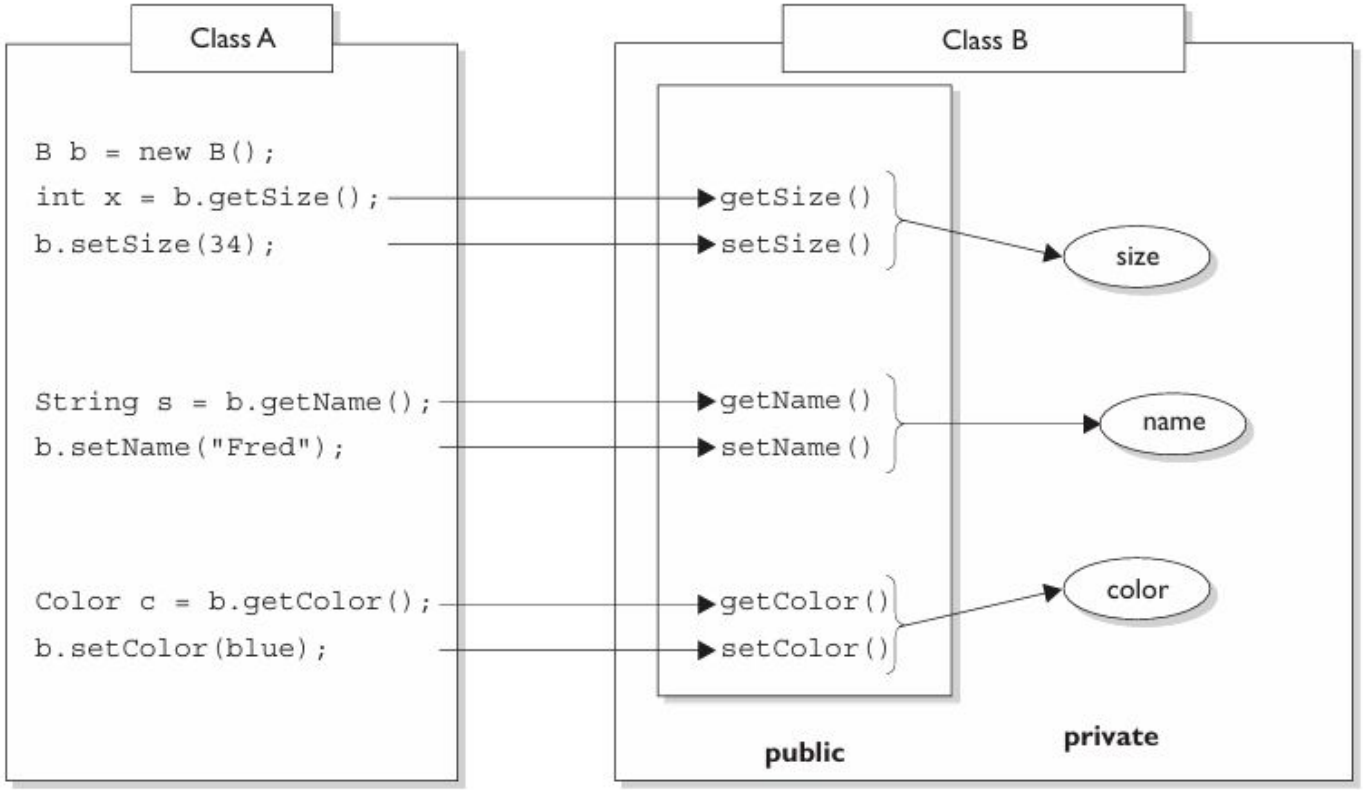
OOP (Recap)

Chapter 2 (Object Orientation)
Book: SCJP

Encapsulation

FIGURE 2-1

The nature of encapsulation



Inheritance

Two most common reasons to use inheritance are -

- To promote code reuse
- To use polymorphism

Inheritance Example

```
class GameShape {
   public void displayShape() {
     System.out.println("displaying shape");
   }
   // more code
}

class PlayerPiece extends GameShape {
   public void movePiece() {
     System.out.println("moving game piece");
   }
   // more code
}


public class TestShapes {
   public static void main (String[] args) {
      PlayerPiece shape = new PlayerPiece();
      shape.displayShape();
      shape.movePiece();
   }
}
```

Inheritance Example

```
class TilePiece extends GameShape {
    public void getAdjacent() {
      System.out.println("getting adjacent tiles");
    }
    // more code
}

  public class TestShapes {
     public static void main (String[] args) {
        PlayerPiece player = new PlayerPiece();
        TilePiece tile = new TilePiece();
        doShapes(player);
        doShapes(tile);
     }

     public static void doShapes(GameShape shape) {
       shape.displayShape();
     }
  }
```

Composition

```
public class Horse extends Animal {
   private Halter myHalter = new Halter();
   public void tie(LeadRope rope) {
      myHalter.tie(rope);  // Delegate tie behavior to the
                           // Halter object
   }
}
public class Halter {
   public void tie(LeadRope aRope) {
      // Do the actual tie work here
   }
}
```

Polymorphism (Overriding)

```
public class Animal {
   public void eat() {
      System.out.println("Generic Animal Eating Generically");
   }
}
class Horse extends Animal {
   public void eat() {
       System.out.println("Horse eating hay, oats, "
                          + "and horse treats");
   }
}
```

Polymorphism (Overriding)

```
public class TestAnimals {
  public static void main (String [] args) {
    Animal a = new Animal();
    Animal b = new Horse();  //Animal ref, but a Horse object
    a.eat(); // Runs the Animal version of eat()
    b.eat(); // Runs the Horse version of eat()
  }
}
```

Polymorphism (Overriding)

```
class Animal {
  public void eat() {
    System.out.println("Generic Animal Eating Generically");
  }
}
class Horse extends Animal {
  public void eat() {
    System.out.println("Horse eating hay, oats, "
                       + "and horse treats");
   }
   public void buck() { }
}
```

```
Animal c = new Horse();
c.buck();  // Can't invoke buck();
           // Animal class doesn't have that method
```

Polymorphism (Overriding)

```
class Animal {
  public void eat() {
    System.out.println("Generic Animal Eating Generically");
  }
}
class Horse extends Animal {
  private void eat() {  // whoa! - it's private!
    System.out.println("Horse eating hay, oats, "
                       + "and horse treats");
  }
}
```

If this code compiled (which it doesn't), the following would fail at runtime:

```
Animal b = new Horse();  // Animal ref, but a Horse
                         // object , so far so good
b.eat();                 // Meltdown at runtime!
```

Polymorphism (Overloading)

```
class Animal { }
class Horse extends Animal { }
class UseAnimals {
   public void doStuff(Animal a) {
      System.out.println("In the Animal version");
   }
   public void doStuff(Horse h) {
      System.out.println("In the Horse version");
   }
   public static void main (String [] args) {
      UseAnimals ua = new UseAnimals();
      Animal animalObj = new Animal();
      Horse horseObj = new Horse();
      ua.doStuff(animalObj);
      ua.doStuff(horseObj);
   }
}
```

Polymorphism (Overloading)

But what if you use an Animal reference to a Horse object?

```
Animal animalRefToHorse = new Horse();
 ua.doStuff(animalRefToHorse);
```

Which of the overloaded versions is invoked? You might want to say, "The one that takes a Horse, since it's a Horse object at runtime that's being passed to the method." But that's not how it works. The preceding code would actually print:

```
in the Animal version
```

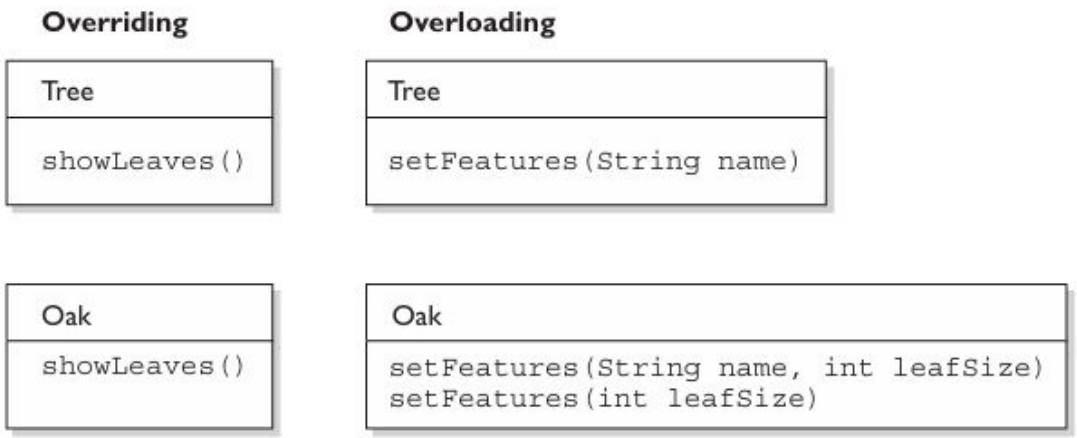
Overloading VS Overriding

Overloading → Compile Time
Overriding → Runtime

Polymorphism (Overloading)

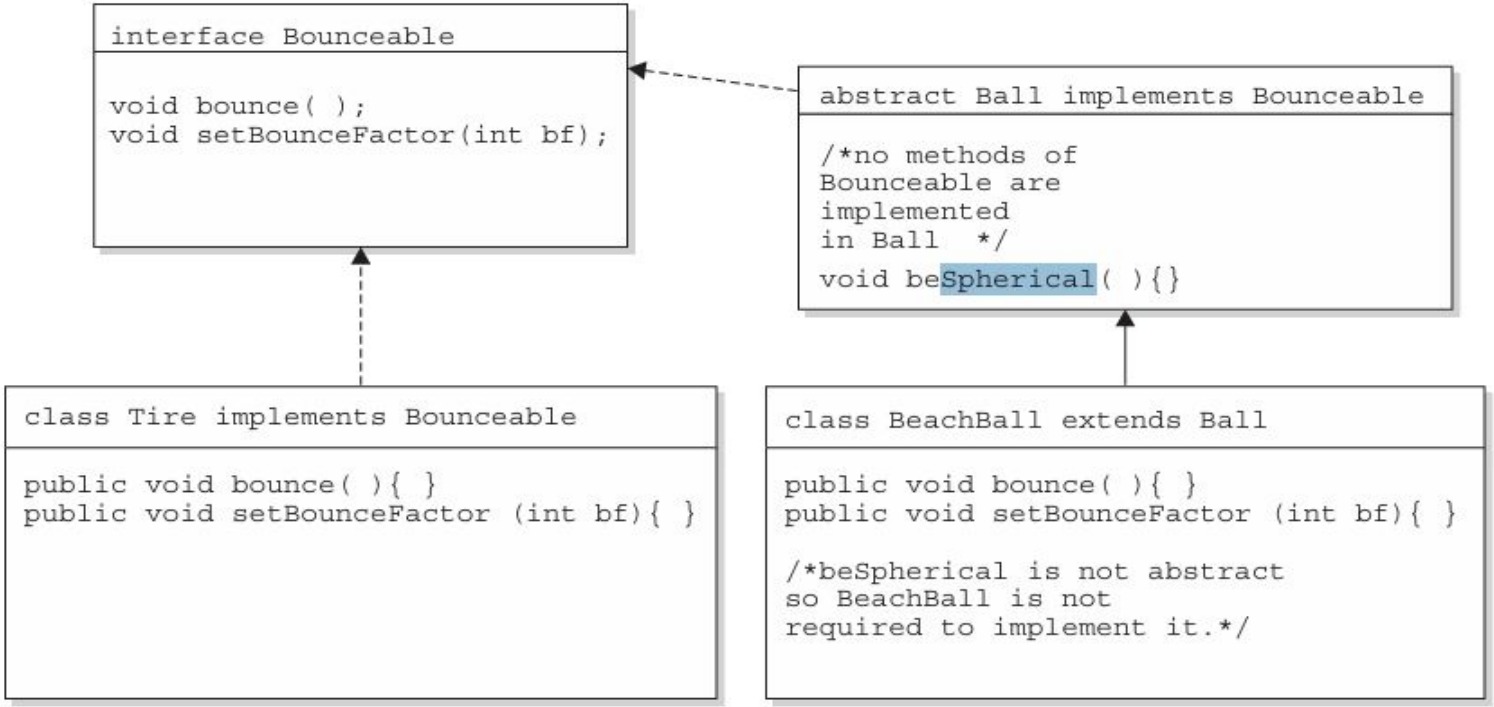
FIGURE 2-4

Overloaded and overridden methods in class relationships



Abstraction and Interface

FIGURE 2-5 Comparing concrete and abstract examples of extends and implements



Constractor (No argument and Parameterized)

```
class Foo {
   Foo() { } // The constructor for the Foo class
}
```

```
class Foo {
   int size;
   String name;
   Foo(String name, int size) {
      this.name = name;
      this.size = size;
   }
}
```

Constractor

```
Foo f = new Foo();  // Won't compile, no matching constructor
```

Constractor

```
Foo f = new Foo("Fred", 43);  // No problem. Arguments match
                              // the Foo constructor.
```

Overloaded Constractor

```
class Foo {
   Foo() { }
   Foo(String s) { }
}
```

Static Variables and Methods

```
class Frog {
   static int frogCount = 0;  // Declare and initialize
                              // static variable
   public Frog() {
      frogCount += 1;  // Modify the value in the constructor
   }
   public static void main (String [] args) {
      new Frog();
      new Frog();
      new Frog();
      System.out.println("Frog count is now " + frogCount);
   }
}
```

Static Variables and Methods

```
class Frog {
   int frogCount = 0;  // Declare and initialize
                       // instance variable
   public Frog() {
      frogCount += 1;  // Modify the value in the constructor
   }
   public static void main (String [] args) {
      new Frog();
      new Frog();
      new Frog();
      System.out.println("Frog count is now " + frogCount);
   }
}
```

Static Variables and Methods

FIGURE 2-8

The effects of static on methods and variables

```
class Foo

int size = 42;
static void doMore( ){
  int x = size;
}
```

static method cannot access an instance (non-static) variable

```
class Bar

void go ( );
static void doMore( ){
    go( );
}
```

static method cannot access a non-static method

```
class Baz

static int count;
static void woo( ){ }
static void doMore( ){
    woo( );
    int x = count;
}
```

static method *can* access a static method or variable

Static Variables and Methods

```
class Animal {
  static void doStuff() {
    System.out.print("a ");
  }
}
class Dog extends Animal {
  static void doStuff() {          // it's a redefinition,
                                   // not an override
    System.out.print("d ");
  }
  public static void main(String [] args) {
    Animal [] a = {new Animal(), new Dog(), new Animal()};
    for(int x = 0; x < a.length; x++)
      a[x].doStuff();                // invoke the static method
  }
}
```

Running this code produces the output:

```
a a a
```

Tightly Coupled Code

```
class DoTaxes {
 float rate;
 float doColorado() {
   SalesTaxRates str = new SalesTaxRates();
rate = str.salesRate;      // ouch
                           // this should be a method call:
                           // rate = str.getSalesRate("CO");
   // do stuff with rate
 }
}
class SalesTaxRates {
public float salesRate;              // should be private
public float adjustedSalesRate;      // should be private
 public float getSalesRate(String region) {
 salesRate = new DoTaxes().doColorado();     // ouch again!
   // do region-based calculations
   return adjustedSalesRate;
 }
}
```

Low Cohesion

```
class BudgetReport {
  void connectToRDBMS(){ }
  void generateBudgetReport() { }
  void saveToFile() { }
  void print() { }
}
```

High cohesion

```
class BudgetReport {
  Options getReportingOptions() { }
  void generateBudgetReport(Options o) { }
}

class ConnectToRDBMS {
  DBconnection getRDBMS() { }
}

class PrintStuff {
  PrintOptions getPrintOptions() { }
}

class FileSaver {
  SaveOptions getFileSaveOptions() { }
}
```